AVL Trees

(**A**delson-**V**elsky and **L**andis Trees)

James Anderson (Updated by Cemil Kirbas)

Data Structures and Algorithms

CS 3100/5100

Wright State University

Some updates are based on the presentation by Mr. Abdul Bari

AVL Tree - Insertion and Rotations on YouTube

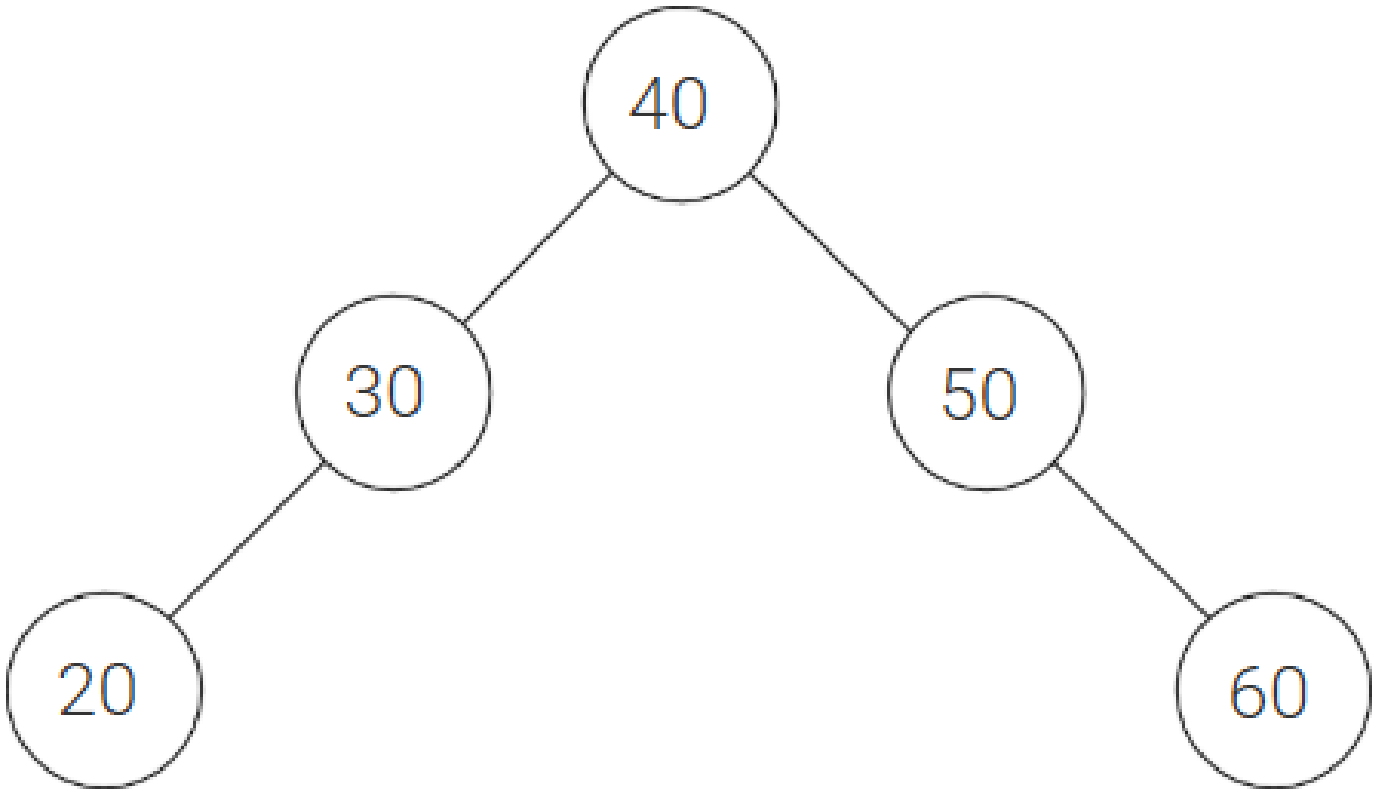
Motivation

- Binary search trees can give us $O(\log n)$ performance
- But – depending on order of insertion, can become $O(n)$
- We could: occasionally take all values in tree, randomize, and re-insert
- Maybe better: want the tree to balance itself when we insert and remove
- We need: a way to determine if a tree is balanced
  - The height of the subtrees can let us know

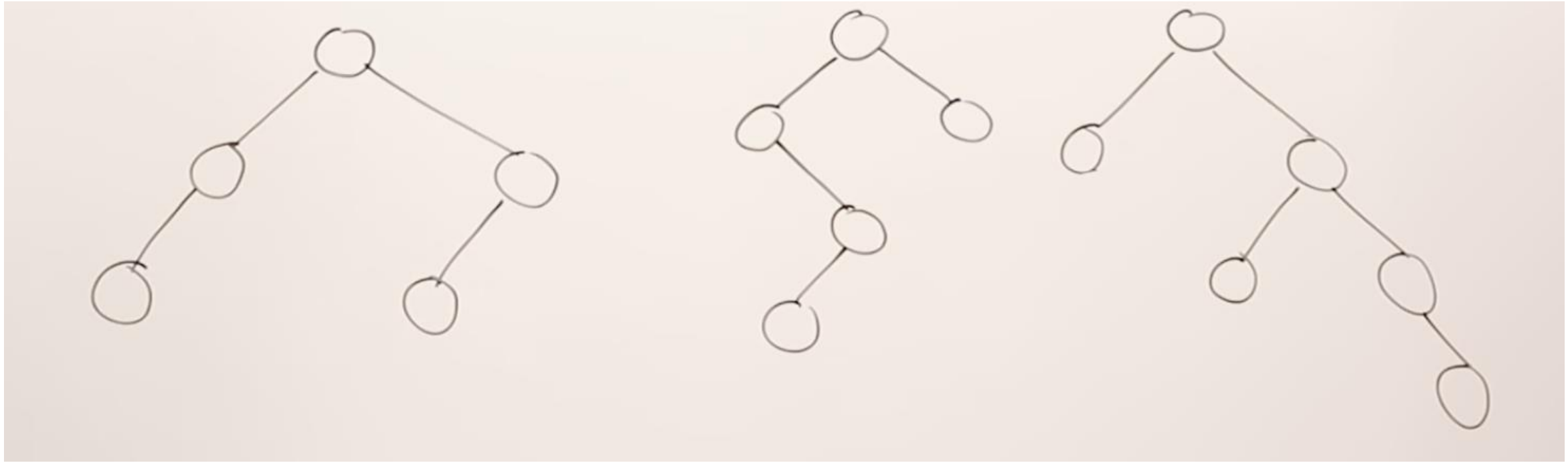
AVL Tree

- **BST + Balance**
- **Height balanced**: for **all** nodes, difference between the heights of left and right subtrees differ by only 0 or 1
- **Balance factor**: left subtree height minus and right subtree height
- Therefore, for all nodes in an AVL tree balance factors must either be **0, 1, or -1**
- If balance of **any** node is **greater than or equal** to **2**, or **less than or equal** to **-2**: entire tree is **out of balance**

Heights and Balances

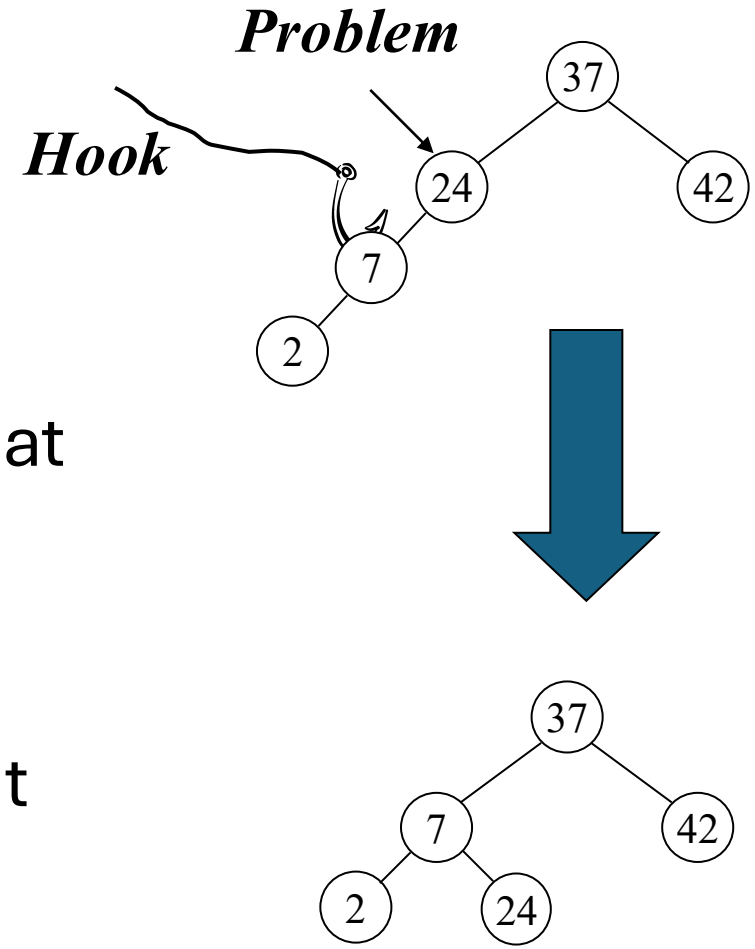


Heights and Balances



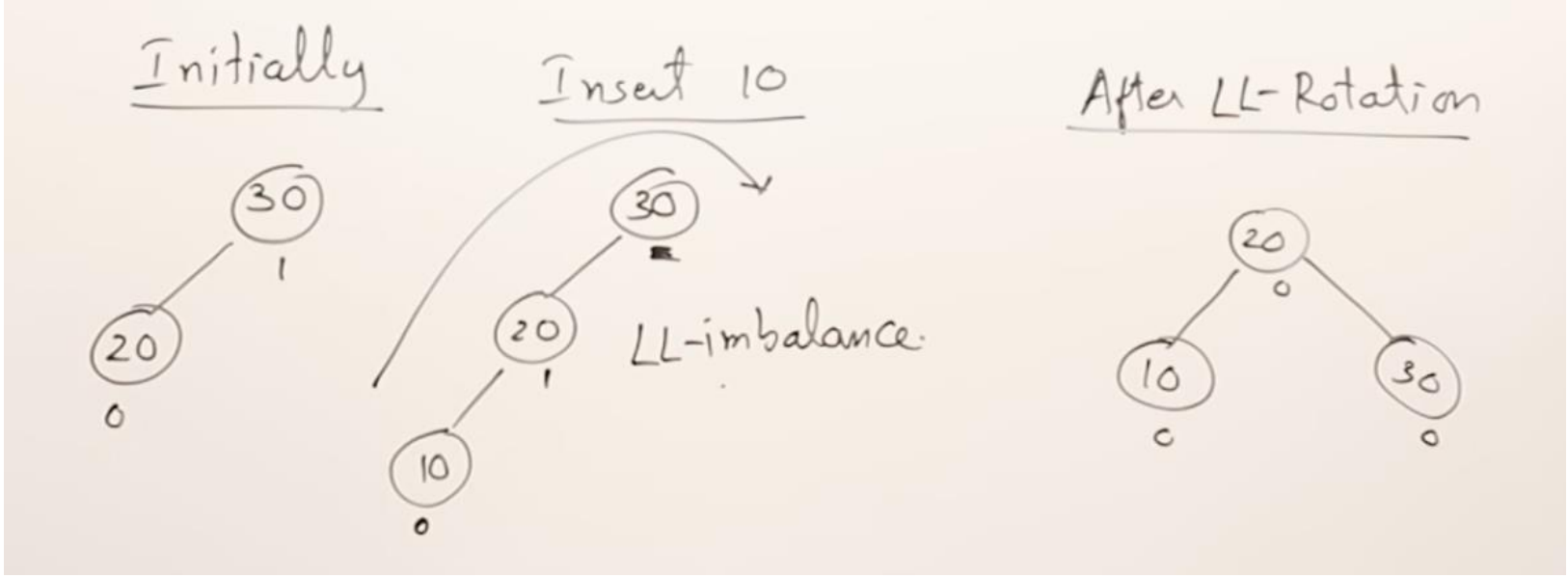
AVL Rotations

- Whenever a node becomes out of balance, we rotate *at* that node
- Happens during insertions and removals
- **Problem** node: the *deepest* node that is out of balance
- **Hook** node: child of problem that is “pulled” up to replace problem
- Rotation keeps values on the correct side of each other



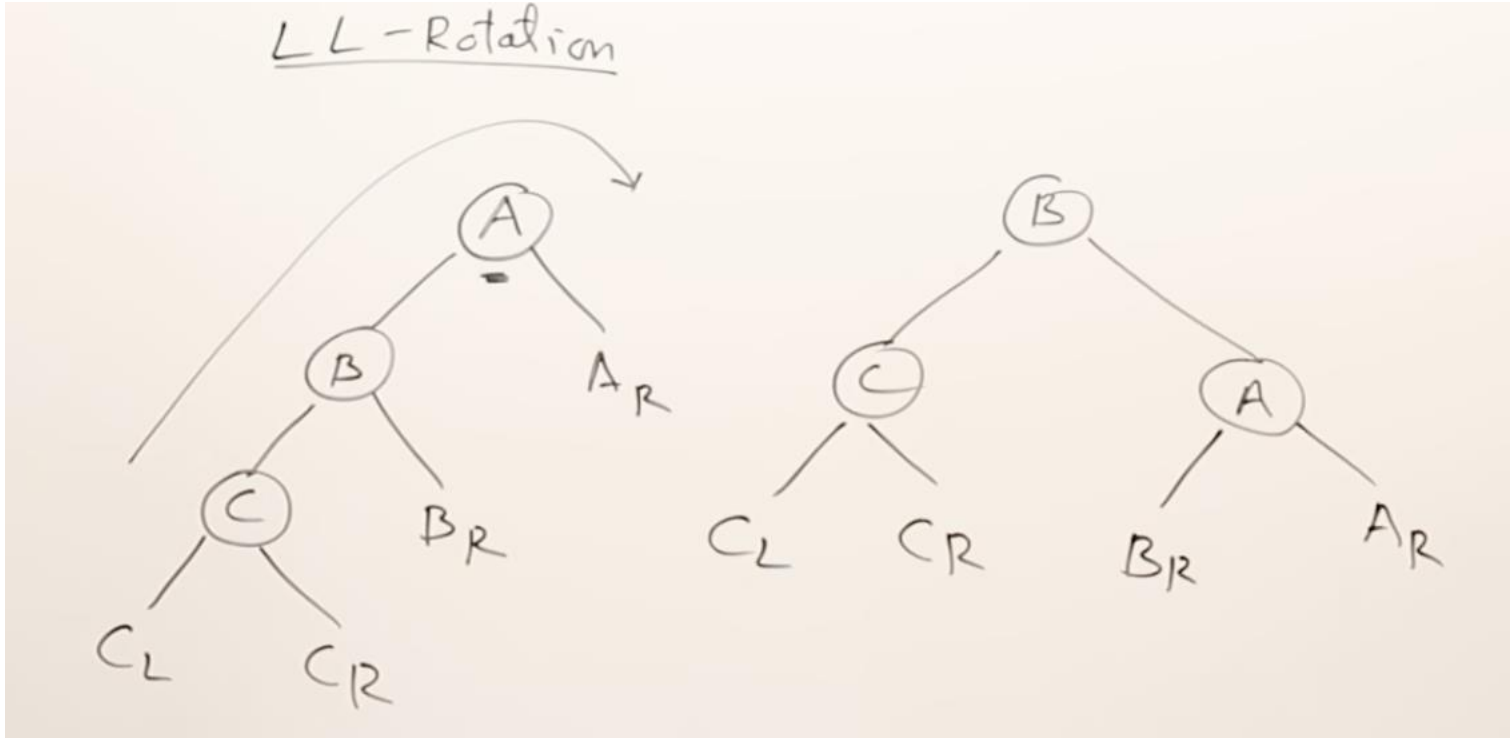
Right Rotation

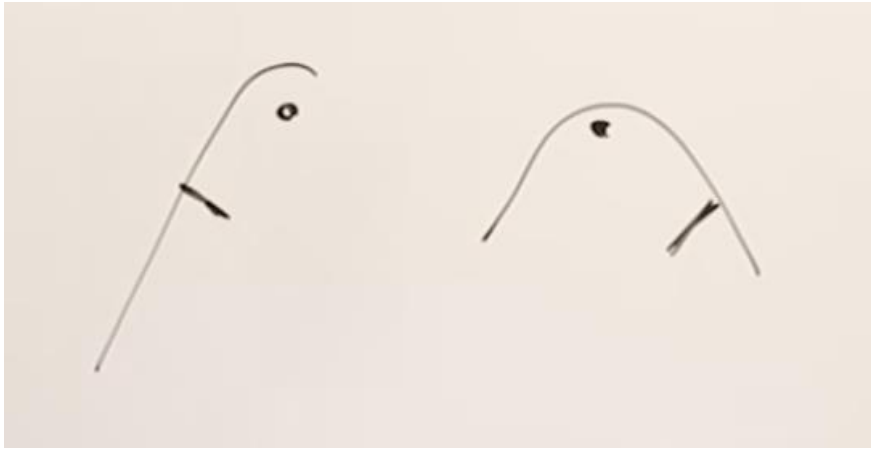
- Problem node: balance of +2
- Hook node: problem node's left child
- Algorithm:
  - Problem drops down and becomes hook's right child
  - Bring hook node up to take place of problem



Right Rotation (con’t)

- What if hook already has right child?
- Problem replaces that child
- Since problem lost it’s left child (hook), put hook’s original right child as problem’s left





Left Rotation

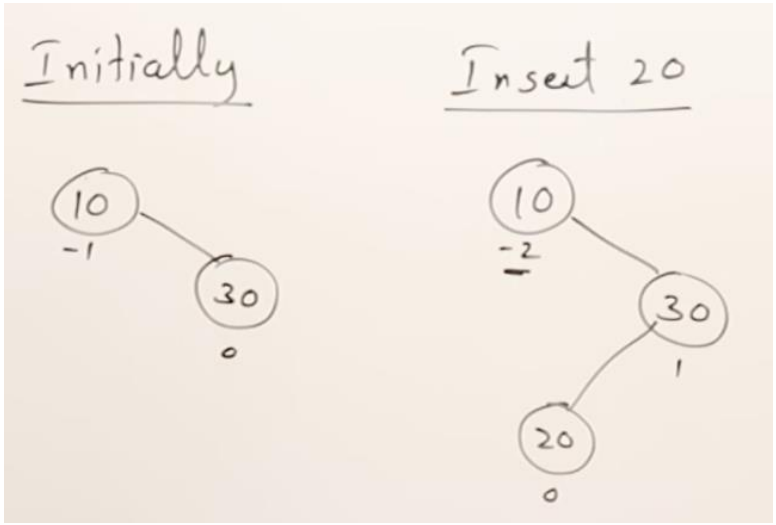
- Problem node: balance of -2
- Hook node: problem node's right child
- Algorithm:
  - Problem node drops down to become hook's left child
  - Hook node takes place of problem node
- If hook has a left child, it becomes problem's right since problem lost right child (hook)

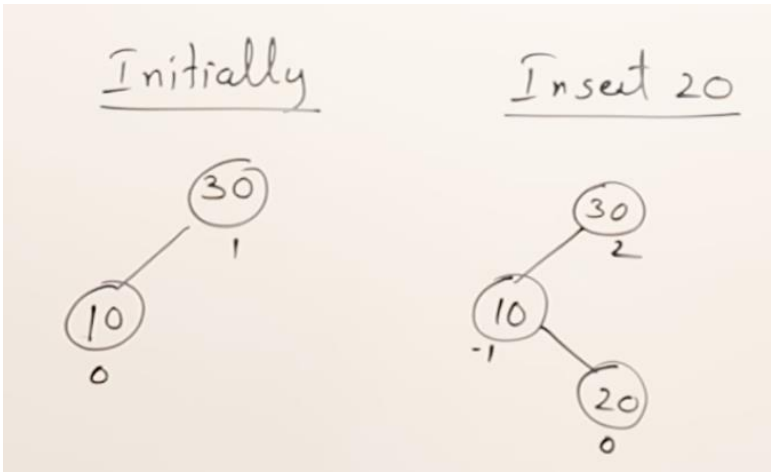
Double Rotation

- Single rotation might not be enough
- Double rotation: either single left then single right, or single right then single left rotations
- First rotation occurs at the hook node, treat hook like the problem node of that rotation
- Second rotation occurs at original problem node

Double Rotation (con’t)

- Double Left rotation:
  - Single Right rotation at hook
  - Single Left rotation at original problem
- Double Right rotation:
  - Single Left rotation at hook
  - Single Right rotation at original problem

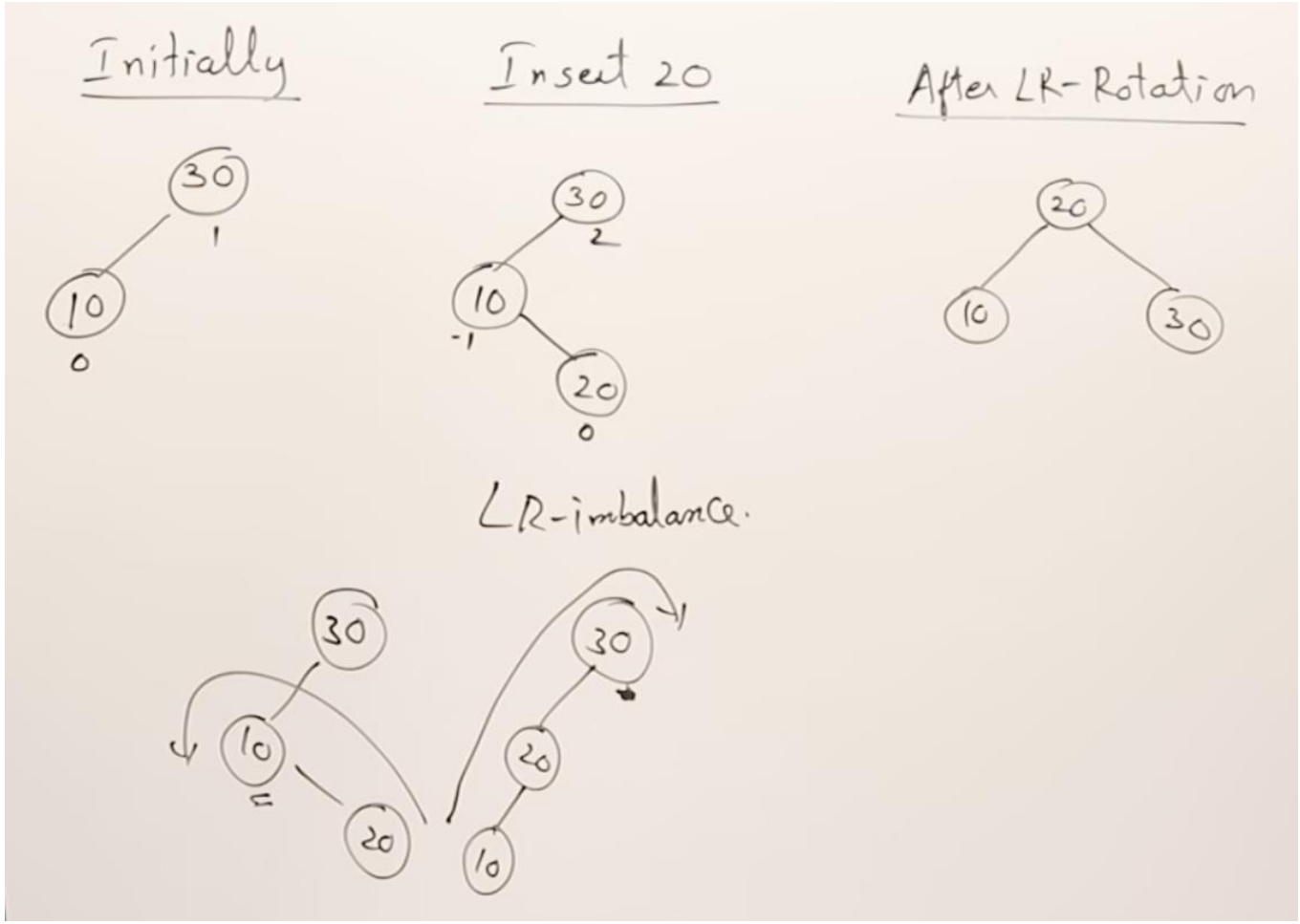




- When?
  - Double Left: problem balance == -2 and hook balance == 1
  - Double Right: problem balance == 2 and hook balance == -1
  - I.e. Problem and hook balance have different signs == Double Rotation

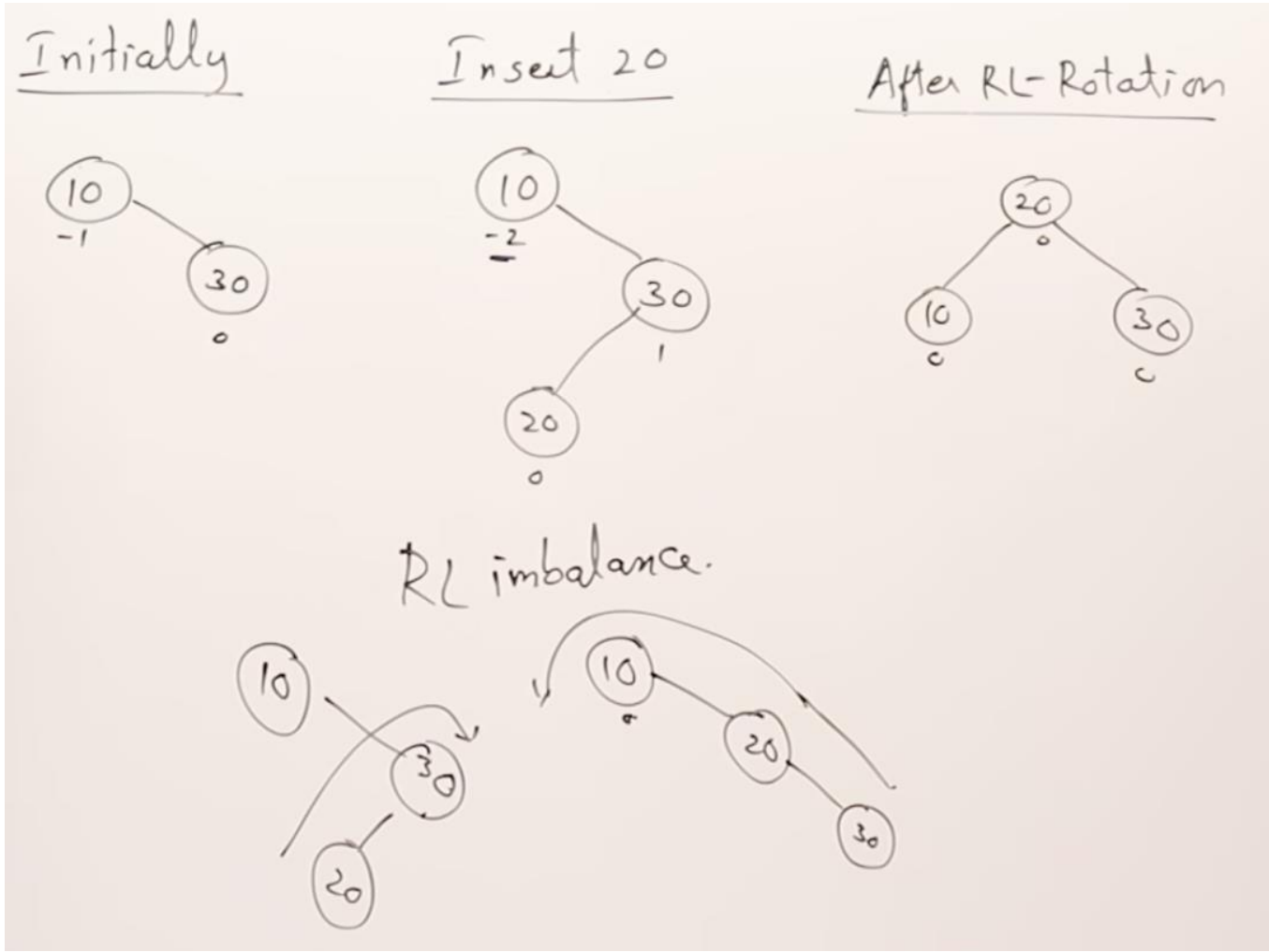
Double Rotation (con't)

- Double Right rotation:
  - Single Left rotation at hook
  - Single Right rotation at original problem
- When?
  - Double Right: problem balance == 2 and hook balance == -1
  - I.e. Problem and hook balance have different signs == Double Rotation



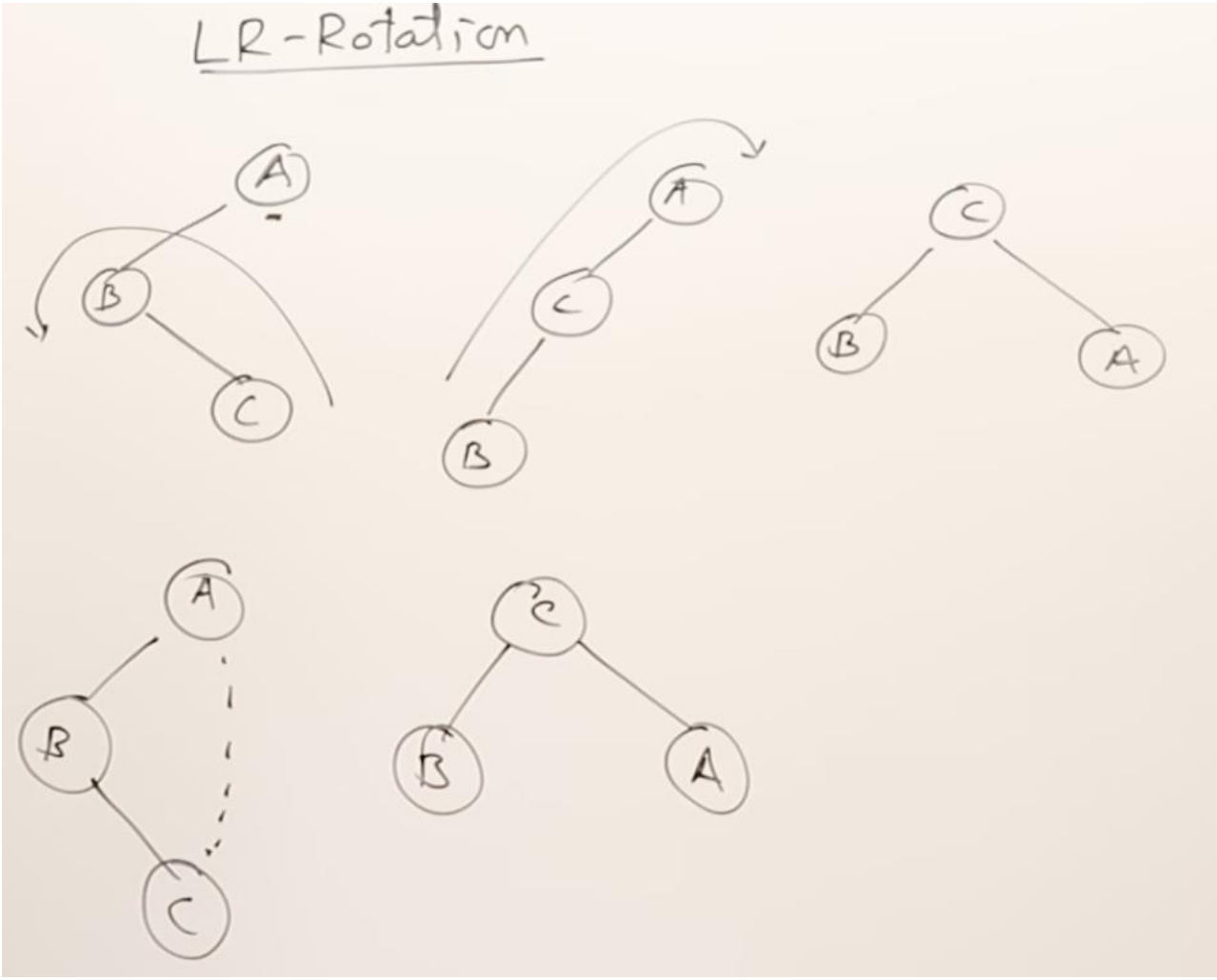
Double Rotation (con't)

- Double Left rotation:
  - Single Right rotation at hook
  - Single Left rotation at original problem
- When?
  - Double Left: problem balance == -2 and hook balance == 1
  - I.e. Problem and hook balance have different signs == Double Rotation



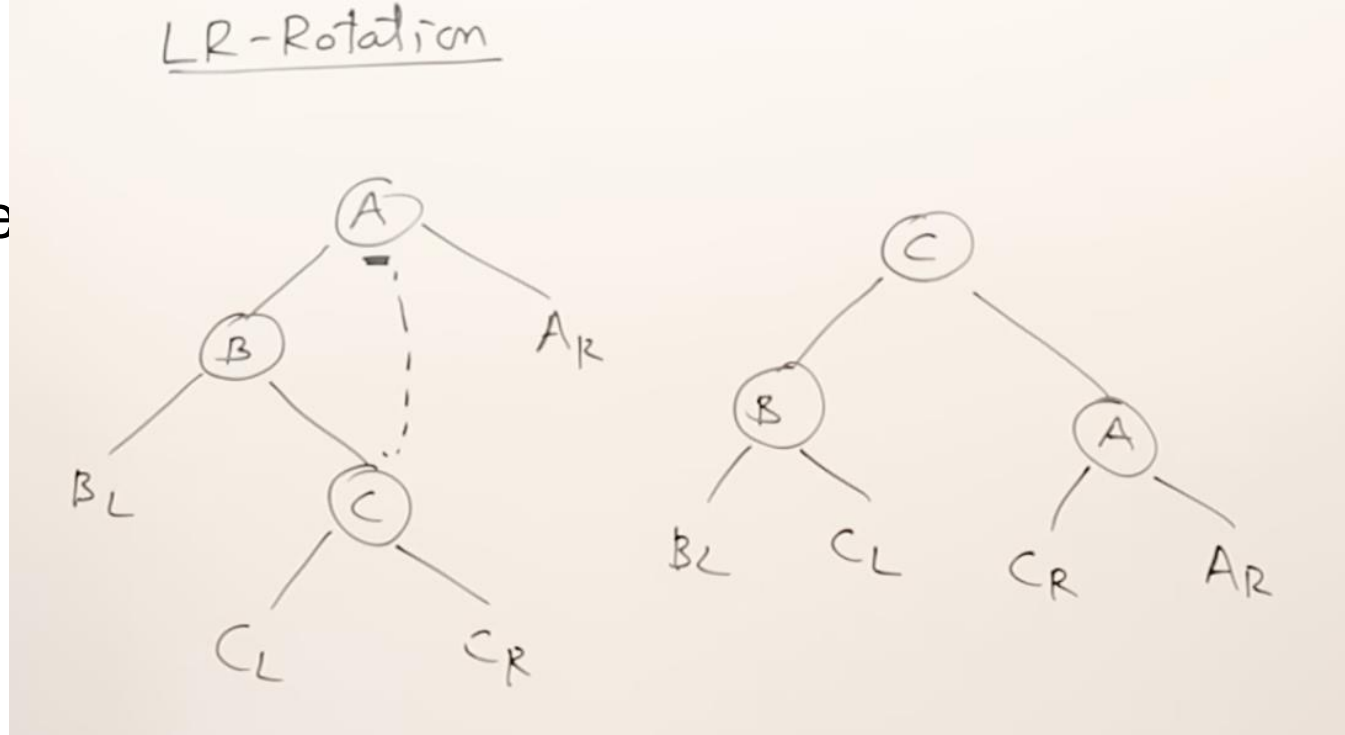
LR-Rotation in Single Step

- Rotation always happens in 3 nodes only.
- **LR Rotation:** Bring the last node in rotation to the top and top node to the right side.
- **RL Rotation:** Bring the last node in rotation to the top and top node to the left side.



LR-Rotation in Single Step

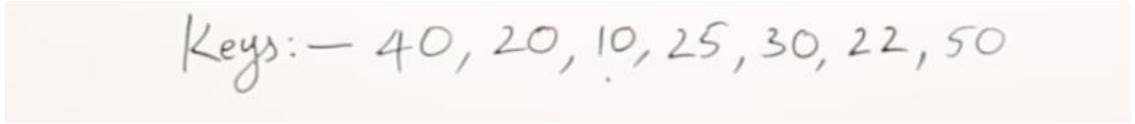
- Rotation always happens in 3 nodes only.
- **LR Rotation:** Bring the last node in rotation to the top and top node to the right side.
- Right child of the last node in rotation becomes the left child of top node.
- Left child of the last node in rotation becomes the right child of the top node's left child.



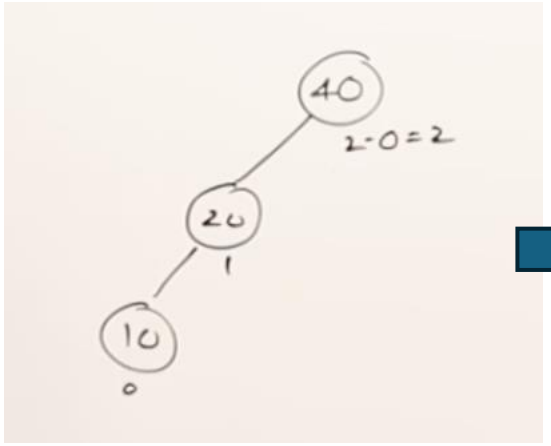
- **RL Rotation:** Take similar action.

AVL Tree Insertions

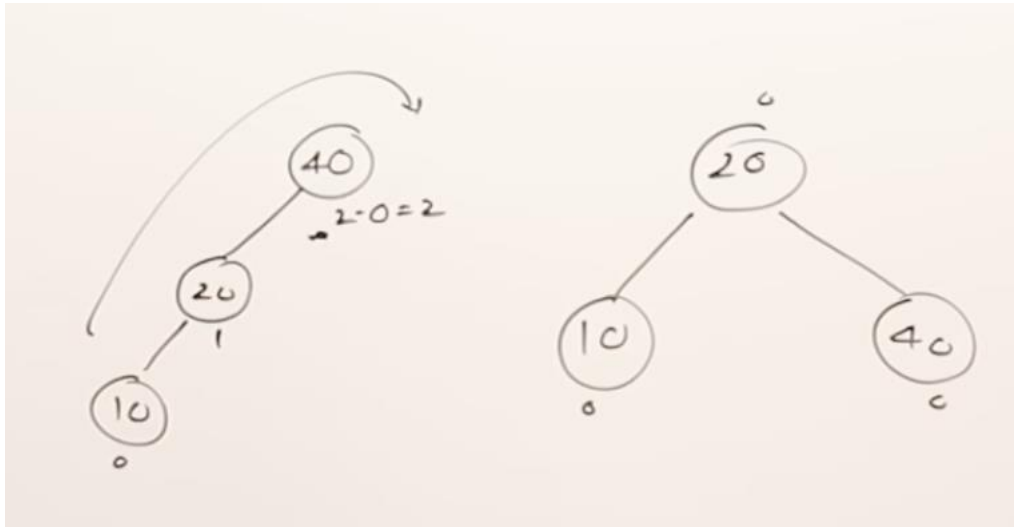
- Insert the keys one-by-one.
- Whenever a node becomes imbalanced, then perform rotation right away and balance the tree.



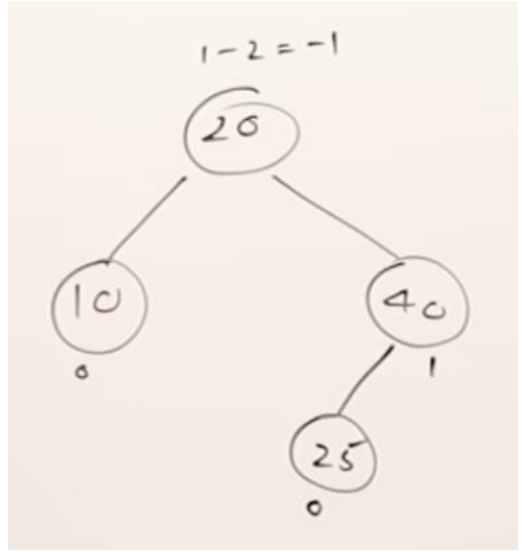
Insert 40, 20, 10



Perform LL Rotation

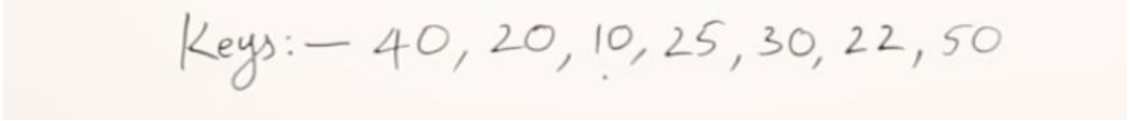


Insert 25. Balanced tree

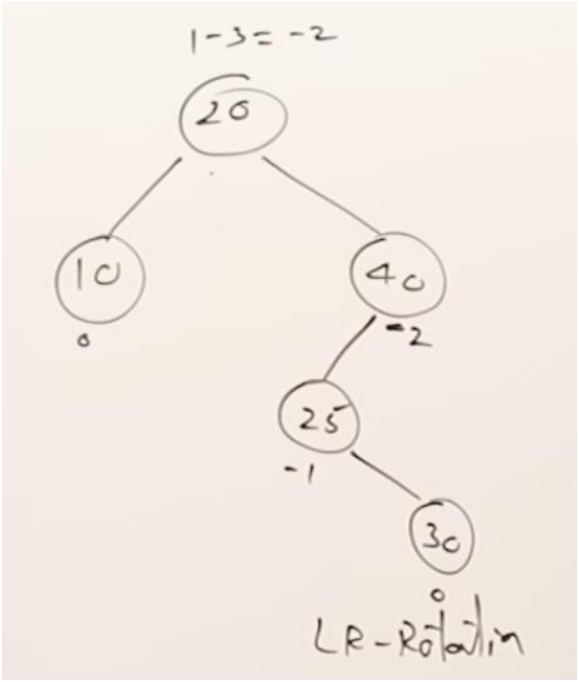


AVL Tree Insertions

- Insert the keys one-by-one.
- Whenever a node becomes imbalanced, then perform rotation right away and balance the tree.

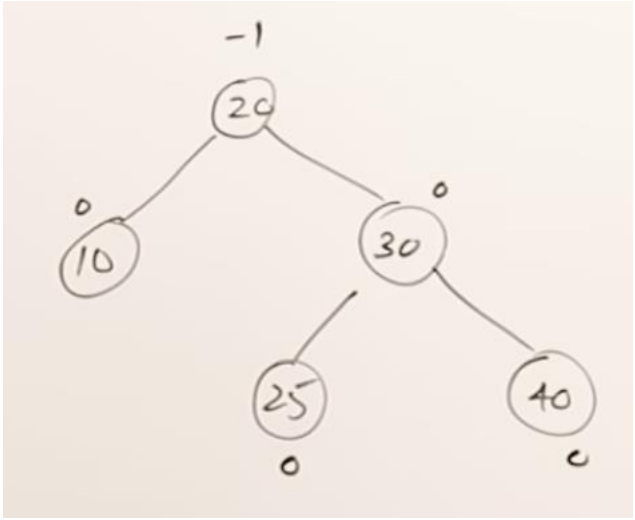


Insert 30. Imbalanced tree



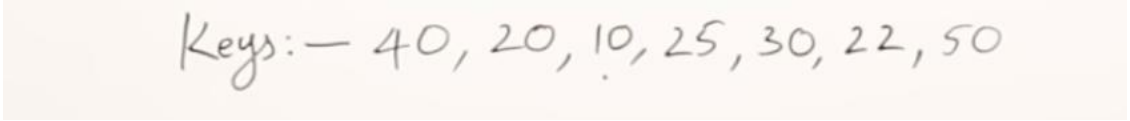
- We have multiple imbalanced nodes.
- Which node to balance?
- Go to the node that caused the imbalance and move up until you find the first imbalanced ancestor.

After LR Rotation. Tree is balanced.

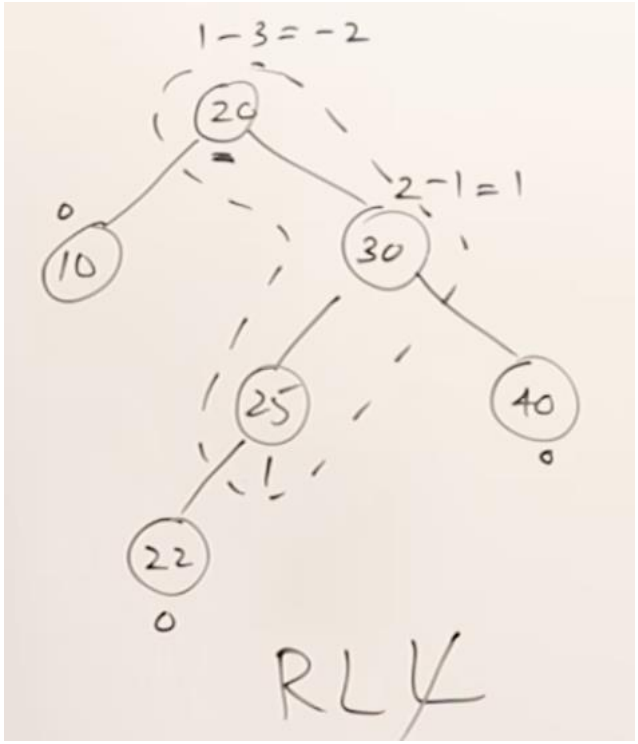


AVL Tree Insertions

- Insert the keys one-by-one.
- Whenever a node becomes imbalanced, then perform rotation right away and balance the tree.

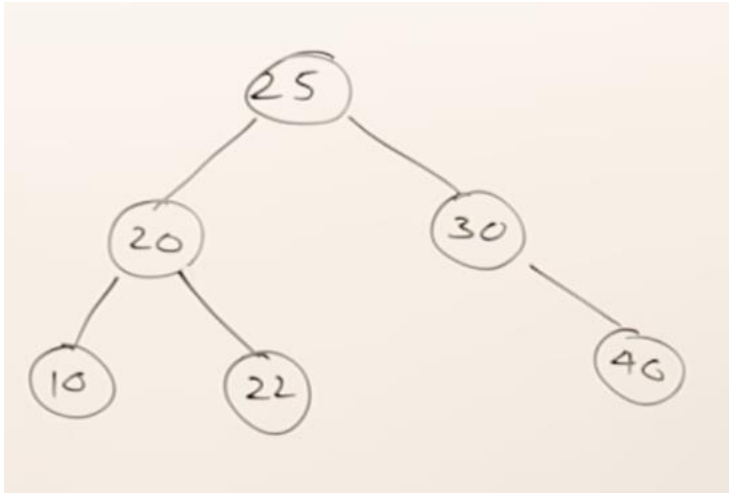


Insert 22. Imbalanced tree



- Only take three nodes including the
- imbalanced node and ignore the rest.

After LR Rotation. Tree is balanced.

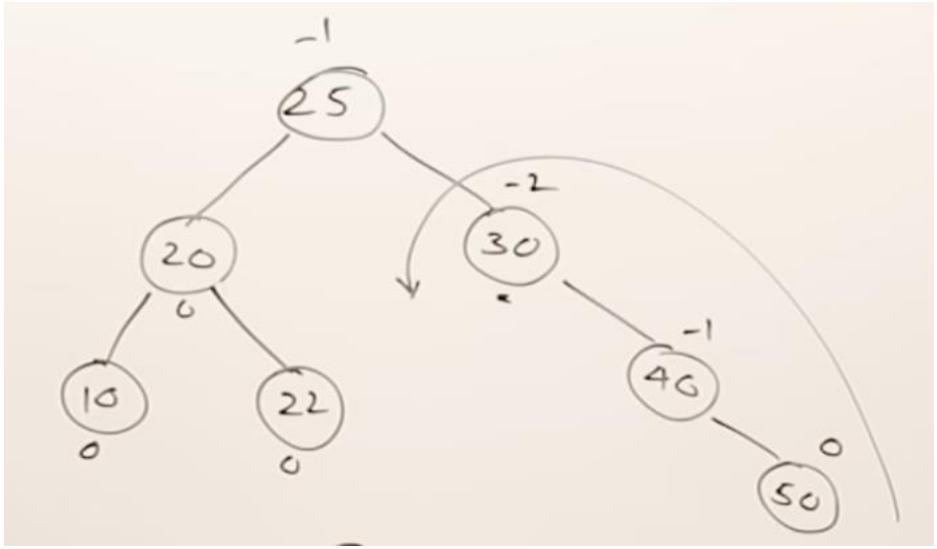


AVL Tree Insertions

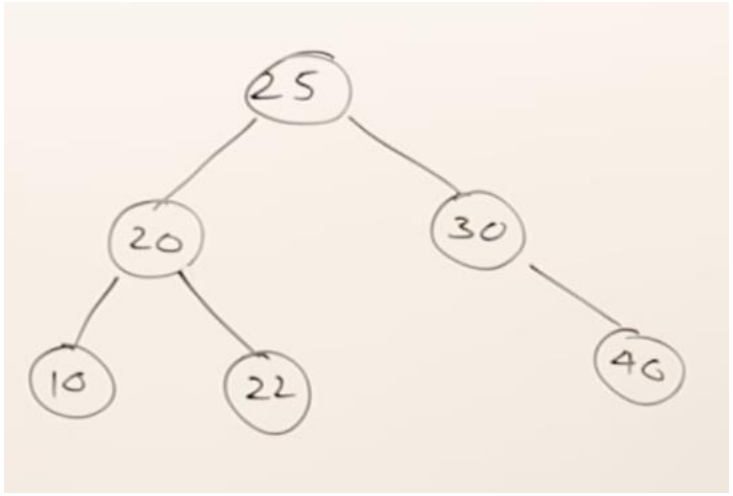
- Insert the keys one-by-one.
- Whenever a node becomes imbalanced, then perform rotation right away and balance the tree.

Keys:– 40, 20, 10, 25, 30, 22, 50

Insert 50. Imbalanced tree



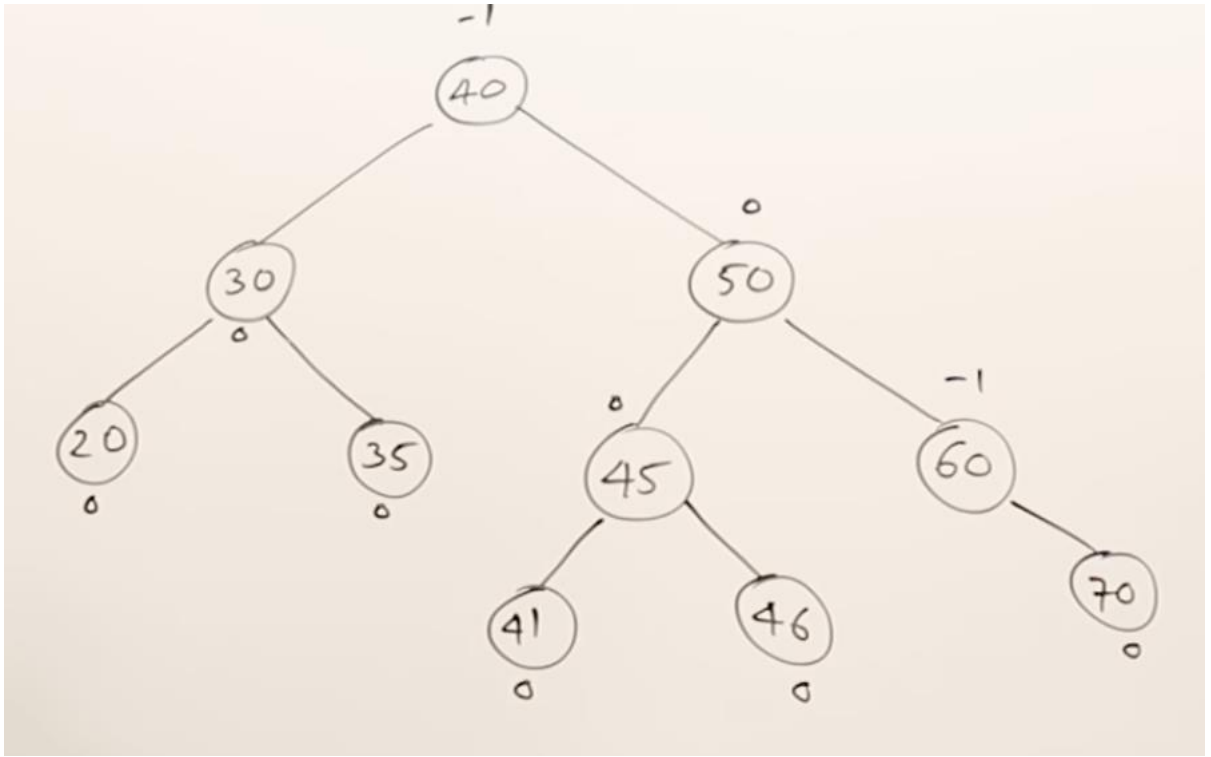
After RR Rotation at node 30.
Tree is balanced.

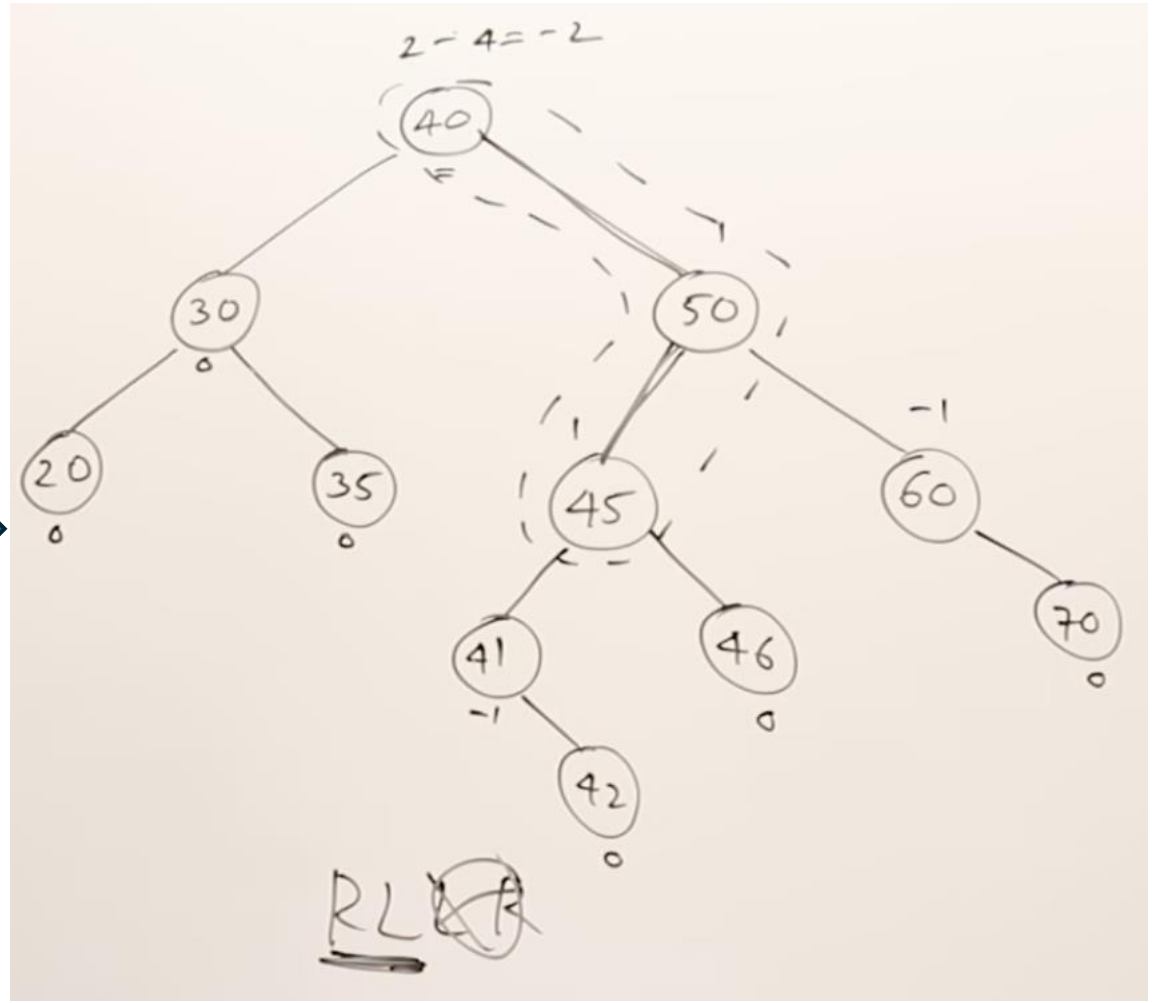


AVL Tree Insertions – Another Example

- Insert key 42.
- Tree becomes imbalanced.
- Perform LR rotation.

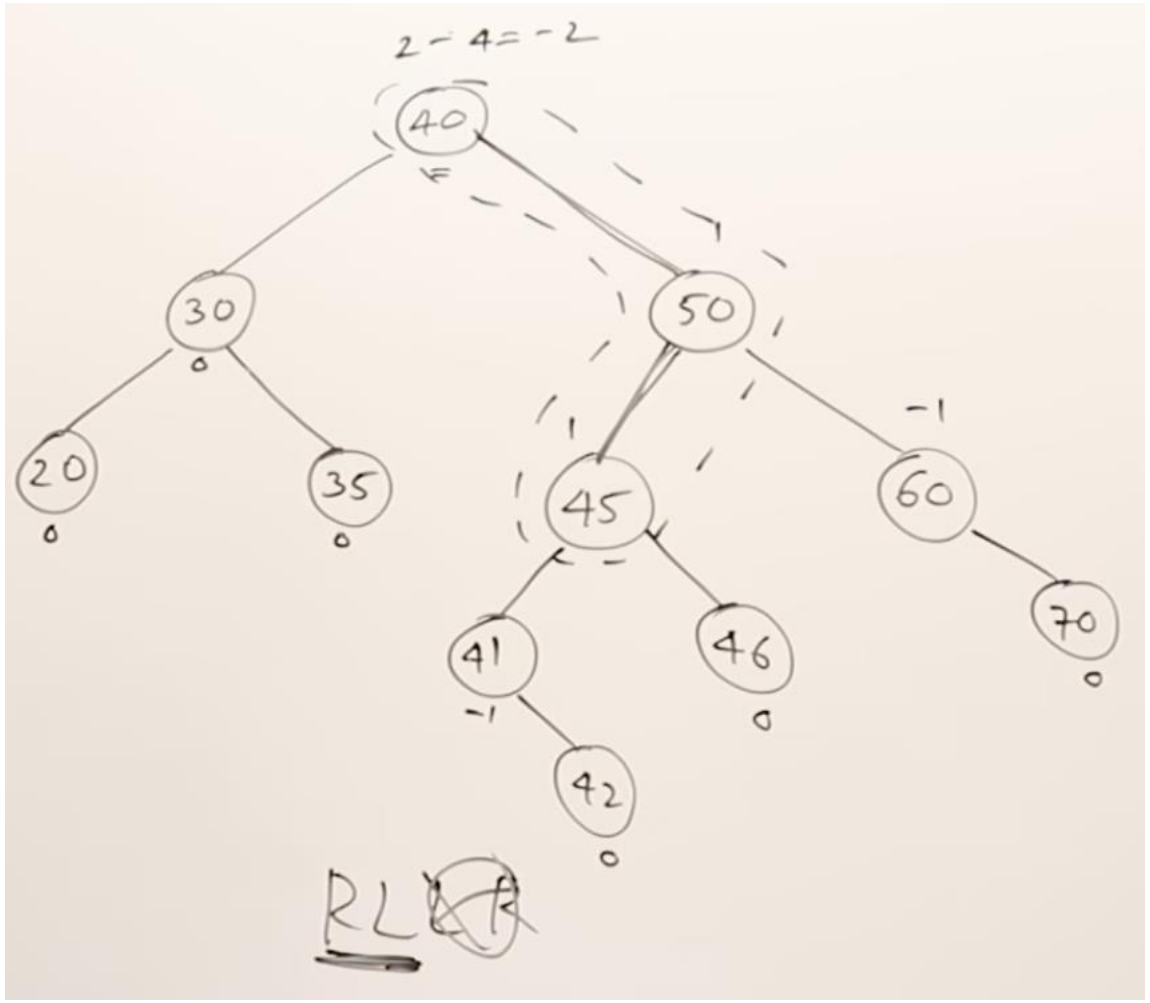
Balanced tree





AVL Tree Insertions – Another Example

- Insert key 42.
- Tree becomes imbalanced.
- Perform LR rotation.



?

AVL Tree Insertions

- First, insert as a regular BST
- After insertion, traverse back up to the root
  - Recursion does this already!
- For each node, update its height and calculate balance
- If we reach a node out of balance: rotate!
- Since tree maintains balance with each insertion and removal, insert will be $O(\log n)$
- Complexity of rotations?

AVL Tree Removal

- First, remove like a regular BST
- Then, traverse back to root
- If any nodes along the traversal are out of balance: rotations!
- Similar to insert, operations keep tree balanced ensuring $O(\log n)$ time complexity for removal

AVL Tree Implementation

- Any helper method need to be $O(1)$ or $O(\log n)$
- If any helper is $O(n)$, complexity for whatever uses it becomes linear
- Example:
  - getHeight that recursively gets height of left and right subtrees
  - Balance method that traverses entire tree to ensure all nodes are balanced

Rotation Summary

Problem Balance	Rotation Direction	Hook Node	Hook Balance	Single / Double
2	Right	Left	1	Single
2	Right	Left	-1	Double
-2	Left	Right	-1	Single
-2	Left	Right	1	Double